

代码大模型基准可信性与执行推理评测

2025—2026 年国际文献详细读后报告

围绕课题：面向代码大模型的基准污染风险诊断与执行推理能力多粒度评测研究

资料核查日期：2026 年 9 月 8 日

研读对象：20 篇 TSE、ICSE、FSE、ASE、ISSTA 论文及其对应公开稿

报告性质：文献研读与研究设计建议；不是已完成的复现实验，也不是对所有最新文献的系统综述。

阅读说明与核心判断

本报告承接此前确定的 20 篇国际论文，暂不混入知网硕博学位论文。选择标准是与研究问题的关联性 & 正式期刊、会议归属，而不是被引次数排名。2026 年论文传播时间短，以被引量判断其重要性并不稳妥。本报告也不宣称这 20 篇构成相关领域的完整文献集合。

证据分为两级：A 为取得公开 PDF 并研读方法、实验、结果或局限等重点章节；B 为官方摘要和元数据层面的研读，不能支持对完整实验的判断。17 篇为 A，文献 [3]、[9]、[12] 为 B。A 不代表逐句精读了全部附录，也不代表已运行作者代码。每篇“评析”“课题连接”均是本报告的分析，不应误写成作者已经验证的结论。

年份以正式期刊卷期或会议年度为主，另行记录预印本年份。TSE 2025 论文可能在 2024 年在线发表，ICSE 2026 论文可能在 2024 或 2025 年已有预印本。截至核查日，ASE 2026 与 ISSTA 2026 尚未召开；相关论文按官方研究论文列表列为已接收 / 待会期，不一概表述为“已正式出版”。

最重要的综合判断是：本课题的价值不应仅落在“发现变体比原题更难”，而应落在一条可以核验的证据链——先确认变体标签与测试有效，再识别原题偏置，随后定位模型预测的执行过程在哪个环节出错，最后用独立数据检验改善措施是否有效。

现有文献提供了三项互补基础：[1]—[3] 帮助区分数据重叠、成员身份与污染效应；[7]—[12] 帮助建立执行过程和解释层面的诊断；[4]、[5]、[13]—[20] 提醒我们同时检查任务定义、测试判据、表面偏差以及实际工作流。不能将这三类问题全部统称为“污染”。

建议优先深入复现的五篇是 REval [7]、AdvPrompt-MIA [2]、LoCaL [15]、Question Neighbourhood [13] 和 PatchDiff [5]。这个次序是针对本课题的主观优先级，不是论文质量排行榜。

一、20 篇论文的研究版图

编号	论文 / 框架简称	归属与年度	主要作用	证据
1	Inter-Dataset Duplication	TSE 2025	可控污染与数据去重	A
2	AdvPrompt-MIA	ASE 2025	扰动式成员推断	A
3	Contamination Means Overestimation?	ISSTA 2026, 已接收	细粒度污染条件	B
4	PAIChecker	ASE 2026, 已接收	需求与参考补丁对齐	A
5	SWE-bench / PatchDiff	ICSE 2026	测试充分性与补丁正确性	A
6	Reproducibility Reflections	ICSE 2026	商用模型实验复现	A
7	REval	ICSE 2025	多粒度运行行为推理	A
8	TraceCoder	ICSE 2026	真实轨迹辅助调试	A
9	VulFaith	ASE 2026, 已接收	漏洞解释忠实性	B
10	SEER	ICSE 2026	推理路径探索与训练	A
11	MGDebugger	ICSE 2026	分层调试与模拟执行	A
12	Causality-Aided Evaluation	ISSTA / PACMSE 2025	提示与生成结果的因果分析	B
13	Question Neighbourhood	TSE 2025	相邻问题与一致性评测	A
14	LLM-as-a-Judge	TSE 2025	评价器本身的可靠性	A
15	LoCaL	FSE 2026	表面相似与功能相似解耦	A, 公开稿题名不同
16	ProxyWar	ICSE 2026	动态竞争环境评测	A
17	COFFE	FSE / PACMSE 2025	正确性与运行效率	A
18	RealisticCodeBench	ASE 2025	实际使用场景的代表性	A
19	AdaptEval	ASE 2025	上下文适配与分层测试	A
20	MCR-Bench	ISSTA 2026, 已接收	多轮审查与状态跟踪	A

必须区分的五个概念

概念	可以观察的证据	不能由此直接推出
数据重叠	训练资料与评测资料存在重复或近重复	模型一定记住了该题
成员身份 / 记忆风险	已知成员对照、受控训练、校准后的推断信号	任一商用模型该样本的真实训练历史
行为鲁棒性	等价改写、相邻问题、上下文变化后的表现	性能下降必然来自污染
功能正确性	规范、测试、差分执行、人工裁决	测试通过等于所有输入都正确
推理 / 解释忠实性	状态、路径或解释因子的干预与验证	能够直接观察模型内部真实计算过程

这一区分决定后续表述的强度。黑盒模型没有训练集真值时，稳妥的结论是“出现与原题偏置一致的异常行为”或“污染风险信号增强”；只有受控实验或其他独立证据支持时，才进一步讨论污染的因果影响。

二、逐篇研读

[1] 跨数据集重复：污染研究应从可验证的数据关系开始

On Inter-Dataset Code Duplication and Data Leakage in Large Language Models。Hernández López 等，TSE 51(1)，2025，192–205；DOI 在线年份为 2024。证据 A，主要核查研究设计、数据构造和有效性威胁。

研究问题与方法：论文考察预训练数据与下游任务数据的交集。作者以 CodeSearchNet 为基础识别重复，再构造包含交集与排除交集的两套训练数据，比较后续任务表现。这比仅观察“公开题做得好”更接近可控的污染效应研究。[论文与公开稿](#)

实验读法：公开稿表 1 报告 CODETRANS、PYTHON-150、TLC 测试集与 CSN 的重复比例分别为 22.40%、14.79%、13.56%；这些是数据重叠比例，不是模型记忆率。训练对照对规模及语言比例作了控制，讨论了微调方式对泄漏影响的调节作用。[正式卷期记录](#)

评析：优点是将“是否含交集”显式操作化，使泄漏讨论具有可核查对象。限制是重复检测所覆盖的相似类型、训练任务和 Java / Python 范围会影响结论；未检出重复不等于没有语义级重复。小规模可控训练也不能直接替代今天大型商用模型的真实预训练过程。

课题连接：应保留原始程序、来源、版本、采样时间、近重复簇和变体父子关系。若设计污染校准实验，干净组与污染组不仅要样本数一致，还应匹配长度、语言、难度、训练步数及答案暴露形式。已有代码大模型可能预先见过旧基准，因此仅从其微调数据中排除某题，不能称为“完全干净模型”。

复现时首先追问：去重单位是函数、文件还是项目？交集样本与替换样本是否难度相当？效果在新来源题目和新变换模板上是否仍然存在？可复用的是受控对照范式，而不是将文中的重叠率作为本课题数据集的先验真值。

[2] AdvPrompt-MIA : 扰动可以形成成员推断信号, 但需要校准

Effective Code Membership Inference for Code Completion Models via Adversarial Prompts。Jiang 等, ASE 2025。证据 A: 核查威胁模型、特征构造、主结果及消融。

研究问题与方法: 输入为代码前缀及真实补全, 目标是判断其是否属于训练数据。AdvPrompt-MIA 对前缀施加五类保持语义的扰动, 比较输出与真实补全, 构造相似性、困惑度等特征并训练分类器。其威胁模型允许攻击者知道少量训练成员, 不是完全无监督的风险探测。[公开全文](#)

实验读法: 表 I 中 Code Llama 7B 上 HumanEval / APPS 的 AUC 分别为 0.97 / 0.95; HumanEval 对应 TPR 0.85、FPR 0.05。AUC 相对提升“约 102%”不是 AUC 超过 1, 也不是提高 102 个百分点。这些数字须连同成员采样和分类训练条件理解。[会议记录](#)

评析: 本研究课题最接近之处在于以行为变化构造信号, 且有跨模型、跨数据集和特征消融。需警惕分类器学习来源、长度或难度差异, 而不是记忆本身; 同源变体若跨越训练和测试划分, 还会造成检测器层面的泄漏。对于只返回文本的接口, 特征获取方式也需逐项核对。

课题连接: 不要直接把“改写后不稳定”定义为成员。论文的出发点反而包含“成员在小幅等价扰动下更稳定”的假设, 而你的非等价变体重点考察“语义应改变时仍回到原答案”。两者的干预对象与预期方向不同, 应该并列成为候选特征, 再由有真值的实验检验。

复现重点: 按原始程序或项目分组切分, 报告低误报区间的 TPR、AUC、阈值和校准误差; 同时设置长度、编辑距离、基础正确率等简单分类基线。若简单基线已取得相近效果, 就不能将复杂检测器的优势解释为“识别了记忆机制”。

[3] 细粒度污染：接触过数据，不等于一定高估能力

Contamination Means Overestimation? A Fine-grained Empirical Study in Code Intelligence. Yang 等, ISSTA 2026, 已接收; 会议尚未召开。证据 B: 目前依据官方摘要, 不对完整统计检验作评价。

研究问题与方法: 论文把污染区分为输入侧、输出侧、非配对及配对四种情况, 覆盖 Java / Python 的生成、翻译和摘要任务, 并比较 RoBERTa、GPT-2 与 LLaMA、StarCoder 等范式。摘要报告: 污染的可见效果受预训练、微调与推断流程影响, 配对污染也不在所有流程中造成显著高估。[官方摘要](#)

评析: 它对本课题的重要性在于限制结论的适用条件, 而不是提供一句“污染无害”的反向口号。没有显著差异可能与真实效应小、样本量、训练强度或评测指标不敏感有关; 这些解释必须读到完整实验后才能区分。本报告不补写摘要未给出的污染比例、置信区间或显著性水平。

课题连接: 将“污染类型”作为实验因素, 至少区分只见题面、只见代码、见输入—输出配对, 以及见原题—推理过程配对。还要区分直接推断和追加训练后的评测, 避免将某一训练流程的结论外推到所有模型。

待全文核查的问题: 污染强度如何定义? “非配对”如何生成? 干净组是否控制训练步数? 预训练模型是否已经接触公开题? 无效应结论是否有足够检验功效? 这些问题未澄清前, 适合将该文作为研究假设细化和反证意识的依据, 不适合作为核心定量比较对象。

[4] PAIChecker : 先确认模型拿到的任务与判分标准是一回事

PAIChecker: Uncovering and Checking PR-Issue Misalignment in SWE-Bench-Like Benchmarks. Wang、Xu、He, ASE 2026, 已接收。证据 A: 公开稿的标注流程、分类体系和局限。

研究问题与方法: SWE-bench 类任务把 issue 当需求, 把关联 PR 当参考答案, 但两者范围可能不同。作者人工分析 SWE-bench Verified 的 500 个实例, 识别 68 个失配实例, 即 13.6%, 归纳五类模式、11 种情形, 再建立以文本判断、代码验证为核心的检测流程。[公开全文](#)

实验读法: 模式包括 PR 超范围、PR 自身有缺陷、规范不完整、后续 PR 以及未说明的字面量要求; 同一实例可有多个模式, 不能简单相加各类计数当总数。论文还在其他 SWE-bench 类数据上评价自动检查器。[会议记录](#)

评析: 优点是把评价错误追溯到任务构造, 而非全部归咎于模型。局限在于“信息是否可获得”取决于工具权限: 人类可访问的讨论和链接, 不一定在被评测模型的输入范围内。因此判定失配时, 必须冻结任务的可见上下文和运行条件。

课题连接: 构造非等价代码后, 如果仍给模型原始功能描述, 究竟要求它遵循描述, 还是预测当前代码执行? 两者可能冲突。执行推理任务应明确以当前代码和输入为准; 生成任务则应同步更新规范和测试。否则所谓“回退原答案”可能只是遵循了未修改的需求。

建议增加任务三方检查: 当前代码、当前要求、当前期望结果彼此一致; 检查者与被测模型解耦。将需求冲突样本单独标记, 不放入污染风险主结果。PAIChecker 是数据质量审核的参照, 不是训练污染检测基线。

[5] PatchDiff : 通过测试的补丁, 也需要更强的正确性检查

Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study. You Wang、Michael Pradel、Zhongxin Liu, ICSE 2026。证据 A: 本报告采用作者站点的 ICSE 终稿。

研究问题与方法: 对 CodeStory、LearnByInteract、OpenHands 在 SWE-bench Verified 上通过原判据的补丁, 作者追加全部开发者测试, 并以 PatchDiff 对模型补丁与参考补丁进行差分测试, 最后人工检查行为差异的性质。[终稿全文](#)

实验读法: 终稿报告平均 7.8% 的候选正确补丁未通过全部开发者测试; 29.6% 表现出与参考补丁不同的行为; 对抽样检查的差异补丁, 28.6% 被判定确实错误。综合结果对应解决率约 6.4 个百分点的高估。不能把 29.6% 直接写成错误率, 也不能把抽样比例当全体逐一核验结果。[复现材料](#)

评析: 最有价值的是将“可疑”和“确认错误”分开, 并将测试判据本身纳入研究。参考实现也可能不唯一, 合法替代实现可以产生未被需求约束的差异。差分测试只能提供反例候选, 最终还需依据规范判断; 没有发现差异也不构成等价证明。

课题连接: 你的非等价变体必须有触发差异的输入。仅修改运算符, 但评测输入未进入相关分支时, 原输出与变体输出可能相同; 此时不能将模型输出原答案计作泄漏。应保留触发输入、首次差异位置、原输出和变体输出的可复查记录。

复现时优先借鉴差分输入生成与不确定性标签, 而不必先复制完整仓库修复流程。将样本区分为已确认差异、测试未区分、任务规范冲突、人工仍不确定, 主指标只使用满足定义的有效样本, 并公开排除数。

[6] 可复现性研究：模型名称相同，不代表实验条件相同

Reflections on the Reproducibility of Commercial LLM Performance in Empirical Software Engineering Studies。Angermeir 等，ICSE 2026。证据 A：复现方法、五项研究结果与障碍分析。

研究问题与方法：作者检查商用 LLM 软件工程研究的材料可用性，并选择能运行的研究进行重复实验，比较原始结果与重复运行分布，而不是仅检查代码仓库是否存在。其复现框架通过容器和透明记录尽量稳定环境。[公开全文](#)

实验读法：最终可开展复现的五项研究中，没有一项所有被考察结果完全复现；两项部分复现，三项未复现。这一结论只对应这五项实际复现实验，不能写成“所有被筛查论文都不可复现”。公开版本对初筛总数存在差异，本报告不以该总数作核心论据。[正式论文 DOI](#)

评析：贡献不只是指出商用 API 的问题，也揭示依赖版本、脚本完整性、参数记录和统计报告的重要性。复现偏差不能自动全部归因于提供商更改模型：环境重建、任务子集和缺失设置也可能改变结果。需要保留从原设置到复现设置的差异清单。

课题连接：把请求时间、完整模型标识、采样参数、最大输出、系统提示、工具权限、重试原因、原始返回和解析器版本一起保存。出现超时、截断或格式失败时，不能只对某类变体追加查询而不记录，这会给它额外预算并改变有效分母。

建议以“原始程序”为统计聚类单位，做配对比较和区间估计；多个变体、多个输入、多个采样不是彼此独立的题目。主表同时给出均值、有效样本数、失败类型和重复运行波动。开源权重模型作为可冻结的基线，商用模型作为外部有效性补充。

[7] REval：本课题最直接的多粒度执行推理基线

Reasoning Runtime Behavior of a Program with LLM: How Far Are We?. Chen 等, ICSE 2025; 公开预印本始于 2024 年。证据 A: 任务定义、数据统计和主结果。

研究问题与方法: REval 要求模型不实际执行代码而预测运行行为, 包含语句是否执行、变量值与类型、执行路径及最终输出四类任务, 并用增量一致性 IC 检查不同层次答案的协调性。评测数据由 HumanEval、ClassEval 改造, 公开稿统计 3,152 个问题。[论文全文](#)

实验读法: 文中平均准确率 44.4% 和平均 IC 10.3 对应其当时的模型集合与设置, 不能当成 2026 年模型的最新表现。一个综合平均也不能说明模型在四类任务上同等薄弱, 应查看分任务结果。[项目与数据](#)

评析: 它将“能写出正确代码”与“能预测代码如何运行”分开, 是本课题的直接依据。需要审慎理解的是层次依赖: 输出正确但路径错误说明外部回答不一致, 却不必然证明模型内部采用了错误路径, 也可能用了另一种算法性计算方式。IC 衡量其定义下的任务一致性, 不是内部认知过程的直接观测。

课题连接: 在 REval 的输出、状态、路径基础上加入原始 / 等价 / 非等价三元组, 观察同一题在语义干预下的错误迁移。真正值得研究的是“输出回退是否伴随关键状态回退”“第一次轨迹分歧是否接近被修改语句”等可检验关联, 而不只是追加几个准确率。

实现难点: 行号会随重构变化, 不能用原始行号逐项比较两段等价程序; 循环、递归会多次执行同一行, 应使用稳定语句标识、调用深度和动态出现次数。类型、别名、原地修改和异常也必须纳入状态语义。先在限制语言子集内保证标注正确, 再扩展复杂程序。

[8] TraceCoder：真实执行轨迹可以帮助修复，但这是工具辅助能力

TraceCoder: A Trace-Driven Multi-Agent Framework for Automated Debugging of LLM-Generated Code。

Huang 等，ICSE 2026。证据 A：总体框架、运行证据及主结果。

研究问题与方法：框架在代码中插入诊断探针，取得真实运行轨迹，再进行故障分析与修复；历史经验机制总结失败尝试，回滚机制按测试表现保留进展。它将运行信息和迭代记忆结合，而非只使用通过 / 失败的二元反馈。[公开全文](#)

实验读法：论文报告相对既有强基线最高 34.43% 的 Pass@1 相对提升；“相对提升”不是增加 34.43 个百分点。这里的 Pass@1 也包含框架内部修复流程，不能与一次无工具生成、相同名字的指标直接公平比较。[会议记录](#)

评析：优点是让调试建立在可验证的运行证据上，并保存失败经验。需要注意，测试上的单调改善不等于程序真实语义质量单调改善；如果反复利用同一测试集，仍可能出现测试过拟合。探针插装对状态、执行顺序和性能的影响也应检查。

课题连接：建议把无工具预测与轨迹辅助诊断拆成两条实验线。前者测模型自身预测执行行为的能力，后者测它利用外部证据的能力。如果把真实轨迹直接放进前者提示，再宣称“执行推理提高”，会使任务失去原定义。

改善实验可设置四组：无反馈、最终输出反馈、关键状态反馈、完整轨迹反馈；固定总 token、调用次数和可见测试。保留独立隐藏测试，才能判断改进是修复能力提升还是针对已见测试的优化。历史记录必须按任务隔离，避免跨题传递评测答案。

[9] VulFaith : 自然语言解释看似合理，不代表解释忠实

Are Reasoning Traces Faithful in LLM-Based Code Vulnerability Detection?. Wang 等, ASE 2026, 已接收。
证据 B: 官方摘要级研读。

研究问题与方法: VulFaith 针对漏洞检测解释, 考察解释是否完整以及各漏洞因子是否忠实, 并通过修改相关因素观察判断是否变化。摘要涉及 10 个开源和 4 个商用模型, 报告部分设置下较高的不完整及不忠实比例。[官方摘要](#)

评析: 这里的 reasoning traces 是自然语言解释, 不是 Python 解释器生成的变量状态日志。将解释中的因素移除后再测, 是一种行为干预思路; 但如果修改同时引入其他安全变化, 或还有未移除的漏洞因素, 模型判断不变未必说明原解释完全无效。需要全文才能评估因素分离与标注质量。

课题连接: 可以要求模型在预测输出后指出决定性条件, 再对条件进行局部干预, 检查其输出是否按预期变化。这比按语言流畅性评分更接近可验证证据。但它仍然测外部解释与行为的一致性, 不能据此声称读出了模型内部的真实思维链。

待核查项包括: 完整性的分母、每因子指标定义、人工标注一致性、漏洞修复的有效性验证, 以及多因子交互是否控制。本报告不将摘要中的最高比例扩展为“所有代码推理均不忠实”, 也不把其漏洞任务结果当成执行预测任务的直接性能预期。

[10] SEER : 改进推理质量比简单延长推理更值得研究

SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning。Gao 等，ICSE 2026；预印本 2025。证据 A：方法概述、训练流程和主实验。

研究问题与方法：SEER 将推理式代码生成视为决策过程，通过带扰动和改进的树搜索探索推理路径，以探索数据训练策略模型与价值模型，再自适应选择直接生成或分步推理。其目标包含避免无效的过度推理。[公开全文](#)

实验读法：在 DeepSeek-Coder、Qwen2.5-Coder 的研究设置下，作者报告 MBPP、HumanEval、LiveCodeBench 上不同配置的绝对提升；摘要范围分别为 4.2—9.3、1.9—9.1、3.5—5.3 个百分点。不能将各数据集最大值相加，也不能把收益全部归于一个组件。[会议记录](#)

评析：优点是将搜索、过程质量和推理长度一起设计。对本课题而言，最重要的限制是终端代码正确并不保证每个中间推理步骤正确：若过程分数主要由最终成功反推，价值模型可能学到与真实执行无关的捷径。搜索和训练引入的额外预算也必须与收益一起报告。

课题连接：若后续开展对比式执行监督，可借鉴“对过程给信号”的思想，但使用解释器验证的状态和转移作为监督，不把另一模型写出的推理文本直接当真值。先证明诊断指标确实能定位错误，再考虑过程训练，能降低课题同时承担数据、诊断和训练三方面风险。

合理的消融顺序为：答案监督、状态监督、路径监督、状态—路径联合监督，以及同等 token 的随机过程文本对照。训练集、验证集和测试集应按原题、来源与变换模板隔离；否则看起来能推广的过程模型可能只学到了重复轨迹形式。

[11] MGDebugger : 分层处理有效, 但模拟执行不能替代执行真值

From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging。Shi、Wang、Wan、Min Wang、Gu, ICSE 2026; 预印本记录始于 2024 年。证据 A: 本次获取的是包含 ICSE 2026 信息的更新公开稿。

研究问题与方法: MGDebugger 将程序分解为有层次的子函数, 自底向上定位与修复错误, 并使用 LLM 模拟执行来追踪子函数状态。其主张是避免把整段程序始终当作单一调试对象。[公开全文](#)

实验读法: 公开稿表 1 的 DeepSeek-Coder-V2-Lite 在 HumanEval 上从不调试的 76.8% 达到 94.5%, 差值为 17.7 个百分点; 该比较不是相对最强调试基线的同等幅度优势。论文还报告 MBPP、HumanEvalFix 和 Defects4J 等实验。[代码仓库](#)

评析: 层次化处理有助于缩短局部诊断上下文, 但分解本身可能改变作用域、闭包、异常或副作用。模拟执行得到的状态仍是模型输出, 可能带有相同的推理错误。需要区分用于启发调试的模拟轨迹和用于评价正确性的真实轨迹。

课题连接: 可将函数、基本块、语句作为不同粒度的诊断界面, 比较信息量与诊断成本; 不宜把粒度越细视为天然越好。细粒度轨迹会增长上下文, 也会增加对齐和标注错误。应同时报告成功率、轨迹长度、调用成本与首次分歧定位精度。

复现重点是拆分后语义保持验证、局部测试和整体回归测试的关系, 以及不同调试方法是否获得相同调用预算。一个可行的课题扩展是比较“真实轨迹提示”和“模型模拟轨迹提示”, 回答改善来自新增可靠信息, 还是仅来自更长的思考过程。

[12] 因果辅助解释：必须先明确干预变量和因果假设

Causality-Aided Evaluation and Explanation of Large Language Model-Based Code Generation。Ji、Ma、Li、Wang、Wang, PACMSE 2(ISSTA), 2025, 1374–1397, DOI 10.1145/3728938。证据 B：官方出版元数据与摘要。

研究问题与方法：论文将提示与代码中的可理解特征结构化，构建因果图，并利用不同提示调整分析输入特征与生成结果的关系，目标是解释代码生成表现并辅助提示改进。[作者机构的出版记录与摘要](#)

评析：其启发是将“什么变化影响了什么”写成明确变量，而不仅展示几个例子。不过，提示长度、问题难度、具体表达方式可能共同变化；学习出的图是否支持因果解释，需要检查干预设计、隐藏混杂和图识别假设。摘要不足以确认这些假设均得到满足。

资料边界：检索到的香港科技大学仓储 PDF 实际为作者博士论文，含同主题章节，并非这篇论文的独立正式全文。本报告没有将两者混同，也没有据该替代材料补写会议论文的完整实验规模。不同页面的模型数量描述存在版本差异，因此不把该数量列作已统一核定事实。

课题连接：应将表面变化幅度、语义变化类型、被修改分支是否触发、输入难度、基础正确率等变量分开控制。代码的控制流图 CFG 描述程序执行结构，因果图描述研究变量之间的假设关系；二者不是同一种图，不能因为都称“图”便直接互换。

后续若使用因果语言，优先从可随机分派的变换策略开始，固定其他条件，明确估计对象是“该干预对某个评测指标的平均影响”。黑盒模型的训练污染不可直接随机化时，应限制为风险关联分析，另用受控模型提供机制证据。

[13] Question Neighbourhood : 同一道模板附近的失败, 比孤立分数更有解释力

The “Question Neighbourhood” Approach for Systematic Evaluation of Code-Generating LLMs. Shahin Honarvar、Marek Rei、Alastair Donaldson, TSE 2025。证据 A: 29 页作者终稿; 不与早期 Turbulence 会议稿重复计数。

研究问题与方法: Turbulence 用参数化题目模板生成一组相近问题, 并同步生成测试判据。终稿包含 60 个模板、每模板 100 个参数实例; 评价准确率、至少生成过正确答案的能力以及多次生成的一致正确性。[作者终稿](#)

实验读法: 邻域内个别参数导致断崖式失败, 可以揭示单题平均分掩盖的泛化不连续。这里的参数变化可能改变任务语义, 不应全部归为“语义等价改写”。同一实例多次生成的一致正确性, 也不同于整个邻域全部答对的比例。[正式 DOI](#)

评析: 优点是把评测单位从孤立样本扩展到受控的问题族。限制是模板覆盖的概念有限, 参数变化还可能增加循环次数、输入尺度和上下文难度。观察到性能不连续, 不能直接认定模型只在背答案; 需要匹配难度和资源后再讨论机制。

课题连接: 你的三元组是小型受控问题族, 可以进一步按变换类型和触发条件扩展邻域。建议保留“同一程序、多种语义保持变换”和“同一程序、多种局部语义改变”两类邻域, 并分别统计, 不把所有扰动混成一个鲁棒性分数。

复现时最值得借鉴的是模板与 oracle 同步构造, 以及以问题族为单位统计。研究创新不能只写“构造更多变体”; 应说明你额外提供了什么: 例如经过动态验证的语义标签、跨变体的状态对齐, 或原答案回退与执行错误之间的定量联系。

[14] LLM-as-a-Judge : 评价器也必须被独立评价

On the Effectiveness of LLM-as-a-Judge for Code Generation and Summarization. Crupi 等, TSE 51(8), 2025, 2329–2345。证据 A; ICSE 2026 Journal First 展示不另算一篇 2026 新论文。

研究问题与方法：作者比较多个模型作为代码生成和摘要评价器的效果，并针对 CoderEval 的测试质量先做检查，例如排除参考实现不能通过测试、空实现却能通过测试的题目，再与模型判断比较。[公开全文](#)

实验读法：即使较强评价器，代码正确性判断与测试真值的一致程度也有限；论文报告最佳总体模型在 Java / Python 上的 Cohen's κ 约为 0.21 / 0.10。这是所研究模型、提示和数据下的一致性，不代表所有未来专用评价器的上限。[正式 DOI](#)

评析：贡献既包括评价结果，也包括评价真值的质量审计。测试本身不足时，模型识别出真实问题反而可能被计为误判；因此“评价器与测试不一致”需要排查哪一方有误。摘要质量与功能正确性是不同任务，不能用前者的较好结果替代后者验证。

课题连接：模型可协助生成变体、建议触发输入、归类错误和初步审阅，但不宜独自确认语义等价、非等价或轨迹真值。主判据应尽量由解释器、测试、静态约束与人工抽查支撑；主实验不要让同一模型既出题、给答案、又判自己的答案。

建议公开错误分类的一致性检查：两名标注者独立判断、分歧仲裁、抽样比例与不确定标签。若资源不足，至少对最影响结论的样本进行人工复核，并报告审核范围，而不是将模型自动标签包装成人工金标准。

[15] LoCaL : 表面像, 不代表功能相同

LoCaL: Countering Surface Bias in Code Evaluation Metrics. Dristi、Dwyer, FSE 2026。证据 A, 但所获取 arXiv 公开稿题为 Analyzing and Mitigating Surface Bias in Code Evaluation Metrics; 以下方法与数字均按该公开稿表述, 不冒称已逐项核对 FSE 终稿。

研究问题与方法: LoCaL 将代码表面相似性与功能相似性拆开, 构造高表面相似但功能不同、功能相近但表面不同等困难代码对。公开稿包含 3,117 对代码, 以差分 fuzzing 支撑功能相似性标注, 并检验四种参考式评价指标。[公开稿](#)

实验读法: 核心不是某一个相关系数高低, 而是评价器在困难区域的失效。若一个评价器倾向于总打高分, 在功能相似子集可能表现不错, 却不能识别功能不同的代码。有限测试估计的是测试分布下的功能相似程度, 不是程序的普遍语义等价证明。[FSE 会议题名记录](#)

评析: 这是本课题非常关键的参照, 因为简单运算符替换可以保持极高文本相似, 却改变行为; 重构则可能相反。还需检查困难样本筛选是否利用了同一批待测指标, 如果在同一数据上用指标误差选择阈值再评价, 可能高估该困难集对新指标的代表性。

课题连接: 建立“表面变化低 / 高 × 语义保持 / 改变”的二维设计。只用重命名代表等价、只用操作符替换代表非等价, 会让类别与变换形式完全重合。应匹配编辑距离, 提供同类变换的多种实例, 并把模板留出测试作为必要检查。

建议的改进是以执行见证支撑每个非等价样本, 并给等价样本标明证明范围: 规则保证、限定语言子集保证, 或仅在测试集合上未发现差异。不要将后者在全文中无条件简称为“语义等价”。

[16] ProxyWar : 动态交互补充静态判分, 但不能自动消除污染

ProxyWar: Dynamic Assessment of LLM Code Generation in Game Arenas。Peng、Wang、Wu, ICSE 2026; 作者顺序按本次公开 PDF。证据 A: 游戏环境、评测机制和污染讨论。

研究问题与方法: 框架让模型生成游戏代理代码, 结合运行测试、迭代修复、竞赛与 TrueSkill 等评分评价实际表现。论文环境覆盖单人谜题、双人棋盘和多人纸牌等九种游戏, 而非仅测试固定输入的输出。[公开全文](#)

实验读法: 竞争成绩反映策略质量、接口遵循和资源约束下的表现, 不能替代语义正确率。对手、赛程、随机性及修复机会都会影响排名; 跨论文的静态 `pass@1` 与游戏评分没有可直接换算的尺度。[代码与环境](#)

评析: 优点是引入行为随环境变化而变化的测量方式。游戏规则和常见策略本身也是公开知识, 动态执行并不等于没有训练暴露。若只比较最终竞赛排名, 难以分清模型是算法好、修复次数多, 还是利用了某一对手的弱点。

课题连接: 可作为外部有效性扩展, 检验在短函数上发现的执行推理弱点是否也影响状态连续演化的任务。但现阶段不建议把完整游戏竞技纳入主线: 它增加环境搭建与统计复杂度, 却不直接解决污染识别的核心证据问题。

更小的可行迁移是增加带持续状态的多步输入序列, 检查模型能否记住状态变化、遵守接口并解释关键转移; 先固定环境和测试种子, 再比较不同变体。其作用是拓宽能力覆盖, 不应充当“天然无污染”的数据来源。

[17] COFFE : 正确性之后, 还应关注执行成本

COFFE: A Code Efficiency Benchmark for Code Generation. Peng、Wan、Li、Ren, FSE / PACMSE 2025, DOI 10.1145/3715727。证据 A: 数据构建、压力输入生成和效率度量。

研究问题与方法: COFFE 在现有题目基础上增加压力测试, STGen 先提取输入契约, 再生成并检查压力输入; 以表达式或生成器减少巨大输入的表达成本。efficient@k 将正确性和效率联系起来, 效率主要通过 CPU 指令计数估计。[公开全文](#)

实验读法: 作者为减少测量波动, 多次执行并对极值作处理。CPU 指令数相对稳定, 但不能完整替代真实运行时间、内存、I/O 或不同硬件的表现。效率测试也必须先确保输入符合题意和参考实现行为正确。[复现材料](#)

评析: 其价值是说明“小输入通过”不能揭示所有能力缺陷。限制包括基于旧基准的继承偏差, 以及为了可评测而选择模型能够完成的题目带来的难度选择效应。若把错误程序因少执行步骤获得的低成本算作优秀, 会出现明显的评价漏洞。

课题连接: 等价变换可能保持输出却改变时间复杂度或轨迹长度。执行推理实验若不给长度和循环次数分层, 可能将上下文超长或计算量增加误判为记忆依赖。需要在变体元数据中记录实际执行步数、最大循环次数和异常 / 超时状态。

优先借鉴的是输入契约与压力输入生成, 而不是马上增加效率优化任务。对每类变换绘制难度分层结果, 确认风险信号不是单纯由运行步数解释。报告总成本时同时区分离线 oracle 构建成本和在线模型推理成本。

[18] RealisticCodeBench : 更难的基准, 不一定更接近实际使用

RealisticCodeBench: Towards More Realistic Evaluation of Large Language Models for Code Generation. Yu 等, ASE 2025, DOI 10.1109/ASE63991.2025.00248。证据 A: 数据来源、场景选择和失败分析。

研究问题与方法: 作者关注开发者实际让模型处理的代码任务, 从带相关使用线索的真实项目中组织需求, 经整理与多语言转化构建评测。其出发点不是持续增加项目依赖, 而是校准基准与现实使用范围的关系。[正式论文公开 PDF](#)

实验读法: 论文比较多个模型在该基准与 HumanEval 上的表现, 并分析参数检查、边界条件等失败。多语言版本源于共同的原始问题, 不能把它们全部视为独立来源样本; 修改需求和参数也不能严格证明所有训练污染都被消除。[会议记录](#)

评析: 最有用的观点是将“真实性”和“复杂度”分离。带有 ChatGPT / Copilot 等显式标识的代码只是实际使用的一部分, 不能代表所有开发者行为; 抽取、改写、语言转换也可能引入人工化特征。真实来源与最终任务真实程度之间仍需要说明构建链条。

课题连接: HumanEval / MBPP 适合做精细控制, 但结论最好在不同来源、包含真实输入约束的程序上重复一次。迁移数据集不是越大越好, 应优先覆盖主数据缺少的能力, 例如字符串格式、边界条件、数据结构和有限上下文依赖。

建议先做小规模分层外部验证: 同一变换协议、同一指标、相近运行步数, 在第二来源上检查信号方向是否一致。若效果只存在于旧题, 应明确报告, 不要把旧题上的检测性能包装为所有代码任务的通用诊断能力。

[19] AdaptEval：模型是否真正遵循了当前改动要求

AdaptEval: A Benchmark for Evaluating Large Language Models on Code Snippet Adaptation。Zhang 等，ASE 2025；对应预印本上传于 2026 年，不因此改为 ASE 2026。证据 A：任务构造、适配分类与分层测试。

研究问题与方法：AdaptEval 从 Stack Overflow 到 GitHub 的代码复用场景构造 164 项任务，提供任务级要求、适配级步骤及两层测试，区分函数接口、逻辑定制与重构等适配类型。论文评测六个指令模型及三个推理模型。[公开全文](#)

实验读法：公开稿给出约 550 人时的构建投入，以及 92.95% 分支覆盖、94.38% 行覆盖。覆盖率高说明测试执行到了较多结构，不等于断言充分。GPT-4o 在所述设置下适配级通过率 72.31% 高于任务级 59.15%，体现整体失败仍可细分。[数据与工具](#)

评析：它把局部改动是否成功与整个任务是否成功分开，是很好的多粒度评价参照。但提供详细适配步骤相当于增加信息，不能把该设置的改善全部解释为模型推理增强。按照人类历史改动还原需求，也可能遗漏开发当时未记录的目的。

课题连接：非等价变体可被解释为“需要忠实执行当前改动”的任务。应区分模型忽略代码变化、误读自然语言要求、理解了修改但执行推断失败三种情况。原答案回退只是其中一种行为，不能覆盖所有失败。

可借鉴它的两层测试设计：整体输出验证与每个受影响关键状态的验证相结合。对多处修改，增加单点修改及组合修改对照，检查模型是否只漏掉其中一项。这样能避免整体二元评分掩盖局部理解能力，也能帮助选择后续过程监督位置。

[20] MCR-Bench : 多轮任务的难点包括状态更新和历史失效

From Static to Dynamic: Benchmarking Real-World Code Review with MCR-Bench. Zheng 等, ISSTA 2026, 已接收; 公开稿 2026 年 8 月上传, 稿内标注 October 2026 卷期。证据 A: 构建流程、状态定义与局限。

研究问题与方法: MCR-Bench 包含 2,269 项、五种语言的多轮代码审查任务; 缺陷以结构化卡片表示, 并跟踪 New、Open、Resolved、Reopened 状态。标注采用局部识别后跨轮归并, 再进行人工核查, 报告 Cohen's κ 为 0.87。[公开全文](#)

实验读法: 多轮评价需要同时判断发现问题、持续跟踪和已经修复后的停止提醒。其“缺陷”分类还覆盖文档、结构、可维护性等内容, 不全部等同于功能错误; 不能将整体结果当成漏洞检测率或代码执行准确率。[数据与代码](#)

评析: 优点是把历史信息纳入可评测状态。需要注意标注者可以查看完整历史, 但评测第 r 轮时模型只能看到截至 r 轮的信息; 后续补丁、评论或最终缺陷状态若进入提示, 会形成时间泄漏。多次标注一致性筛选也可能排除最困难的样本, 作者在局限中承认了这一点。

课题连接: 多轮修复或诊断中, 模型可能坚持已失效的原假设。这与单次非等价变体中的原答案回退在行为上相似, 但历史提示偏置和预训练记忆是不同来源。可先研究可控的会话历史偏置, 再检验是否能与无历史条件下的风险信号区分。

建议将该文作为动态扩展而非主基线。每次修改都记录代码版本、当前可见信息、缺陷状态和模型依据, 使用新会话 / 保留历史的配对设置。由此回答模型究竟无法理解当前代码, 还是被旧上下文误导。

三、跨论文归纳：已有结论与尚未解决的问题

3.1 三条相互补充、不可混同的研究线

第一条线是“评价是否可信”：数据重叠 [1]、污染条件 [3]、需求失配 [4]、测试不足 [5]、评价器偏差 [14] 和复现障碍 [6] 都可能让分数失真，但发生位置不同。即使彻底排除训练污染，规范冲突和弱测试仍会影响评价。

第二条线是“模型能否跟踪语义”：REval [7] 测运行行为，LoCaL [15] 检查评价器是否被表面形式误导，Question Neighbourhood [13] 测问题族中的泛化；三者共同支持在控制条件下考察局部变化，而不是只看孤立平均分。

第三条线是“额外过程信息能否帮助”：TraceCoder [8] 用真实执行，MGDebugger [11] 用模拟执行与层次分解，SEER [10] 用搜索与过程训练。它们改善不同 workflows，不能仅凭更高的同名指标判定哪一种“推理机制”更强。

3.2 适合借鉴的设计与迁移边界

设计来源	可以直接借鉴	迁移时必须补充
[1][2][3] 污染与成员推断	干预对照、行为特征、校准	已知成员真值、基础模型暴露边界
[7] REval	输出、状态、路径分层评测	变体间事件对齐、动态出现次数
[13][15] 邻域与表面偏差	问题族、语义与形式二维控制	编辑距离与难度匹配、模板留出
[4][5][14] 判据审核	需求对齐、差分测试、人工复核	不确定标签和样本排除流程
[8][10][11] 过程改善	分级反馈、过程学习、历史管理	等预算、隐藏测试、无工具对照
[16]—[20] 场景扩展	动态状态、适配要求、真实来源	独立来源及时间边界验证

3.3 本报告提出的研究缺口

以下是基于这 20 篇形成的研究判断，不是对全领域“首次性”的证明。

缺口一：风险检测与执行诊断之间缺少明确衔接。成员推断告诉我们样本可能被见过，多粒度评测告诉我们运行行为预测可能出错；本课题可研究二者何时相关、何时不相关，而非预设所有执行错误都来自记忆。

缺口二：跨变体的过程对应关系不够容易获得。原程序和重构程序的行号、变量名和执行步数可能不同。能够输出可审核的状态 / 事件对应，并明确不可对齐的情形，比单纯扩大变体数量更有方法价值。

缺口三：风险指标的反例检验不足。原题正确、变体错误，可能来自难度、规范冲突或解析失败。若一套指标能通过这些反例检验，并在有污染真值的校准集上成立，才具备较强解释力。

缺口四：诊断结果是否指导改进仍需验证。增加过程监督后，必须考察未见原题和未见变换上的效果，并检验原题偏置是否减弱；只提高训练同源变体的准确率，不能证明推理能力得到普遍增强。

四、对开题方案的具体建议

4.1 将三个研究问题写成可证伪命题

RQ1: 在语义标签和输入有效性经过验证的受控代码变体上，模型是否表现出不能由编辑幅度、运行复杂度和基础难度充分解释的原题偏置？主要依据 [1][2][3][13][15]。输出应是行为测量及有真值场景的检测效果，而不是直接给每个商用模型判定“污染率”。

RQ2: 上述异常是否伴随可定位的状态、分支或路径预测错误？首次分歧是否与语义修改点相关？主要依据 [7][9][11][19]。输出应包括分层准确率、跨变体一致性、首次分歧和错误类别，并保留不支持预设机制的反例。

RQ3: 以经过执行验证的关键状态或转移作为监督 / 反馈，能否在独立题目与独立变换上改善性能并降低异常回退？主要依据 [8][10][11]，质量控制依据 [4][5][6][14]。如果资源有限，先做反馈式改善，再决定是否开展训练。

4.2 最小可行实验流程

阶段一：冻结程序来源、解释器、依赖及任务类型。优先限定确定性 Python 函数，明确输入域；暂不纳入网络、随机性、文件系统和依赖外部时间的函数，避免 oracle 不稳定。

阶段二：为每个原程序构造等价候选与非等价候选，记录操作规则和修改位置。生成变体的模型与评测模型分离；保留生成失败和过滤日志，不只留下“看上去符合预期”的例子。

阶段三：校验语法、输入合法性、终止、原参考实现与测试的一致性。对非等价候选寻找区分输入并复核；对等价候选记录具体保证范围。不满足主任务定义的样本进入附表，不混进主结果。

阶段四：离线运行原程序和变体，保存输出、异常、关键状态与轨迹作为隐藏真值。无工具主实验不给模型这些执行结果；模型只接收当前代码、输入及明确的输出格式要求。

阶段五：在固定预算下随机化题目顺序，采集三类代码的预测；固定解析协议，把解析失败、截断、拒答、超时分别记录。对格式修复重试设置统一规则，不进行选择性追加提问。

阶段六：按原始程序进行配对统计和聚类区间估计，再按变换种类、执行步数、分支条件及数据来源分层。最后用人工复核的反例检查解释，并进行独立来源验证。

4.3 建议指标：定义、分母和边界

以下指标是本报告针对课题提出的操作化建议，并非宣称已被上述论文统一验证。

记原程序为 P ，等价候选为 E ，非等价候选为 N ，输入为 x ；真实可观察结果分别为 y_P 、 y_E 、 y_N ，模型预测为 $\hat{y}_P$ 、 $\hat{y}_E$ 、 $\hat{y}_N$ 。结果可包含返回值、必要的输入原地修改及异常状态；是否包括 `stdout` 必须在协议中固定。

基础正确率：分别计算三类程序的预测正确率。比较原始与等价表现时必须使用配对有效样本，而非分母各异的总体均值。对“等价”标签仅经有限测试支持的情况，结果应注明测试域限制。

等价联合正确率：在有效等价样本对上，统计 $\hat{y}_P = y_P$ 且 $\hat{y}_E = y_E$ 的比例。另报预测一致率，但不能用一致率替代正确率，因为两个版本都答错也可能一致。

非等价语义响应正确率：在 $y_P \neq y_N$ 的区分输入上，统计 $\hat{y}_N = y_N$ 的比例；再单独报告原程序也答对时的条件结果。区分输入集合大小应公开，未触发变化的输入不能放入此分母。

原答案回退率（Original-leak，行为指标）：在集合 $D = \{(P, N, x) : y_P \neq y_N \text{ 且 } \hat{y}_P = y_P\}$ 上，计算满足 $\hat{y}_N = y_P$ 的样本比例。 D 为空时记为不适用。该量描述“原来能答对、语义变化后仍答原输出”的行为，不等于训练集泄漏概率。

为避免遗漏，应同时报告：全部区分输入上的回退率、条件回退率，以及错误预测中回退所占比例。它们分母不同，不能只保留最显著的一项。若输出空间很小，还应与多数答案、基础概率和随机对照比较。

状态 / 路径指标：按确定的动态事件位置检查变量值与类型、分支选择、执行事件序列。首次分歧位置和前缀正确长度能提供比全序列完全匹配更细的诊断。归一化长度指标必须给出短轨迹处理方式，避免长程序被系统性惩罚。

检测指标：只有在已知成员或受控污染标签的数据上，才能报告检测器 AUC、TPR@FPR 等，并将变体族整体放入同一划分。黑盒真实模型上的风险分数应与校准集检测性能分表，不能把一个阈值未经验证地跨模型通用。

4.4 必须做的对照和消融

对照	要排除的替代解释	具体做法
触发 / 不触发输入	代码改了但行为未改变	同一非等价变体配两类输入
表面幅度匹配	检测器只识别编辑长度	在相近编辑距离内比较语义类型
运行复杂度匹配	轨迹变长导致推理失败	按执行步数、循环次数分层
规范一致性	模型只是遵循原题面	无原描述与同步更新描述的对照
新会话 / 带历史	旧上下文导致回退	隔离预训练风险与会话记忆
已知污染 / 已知排除	风险分数没有校准	构造可控暴露实验，并说明基础模型边界
简单特征基线	深模型学习数据偏差	长度、相似度、难度分类器
新原题 / 新模板	训练中见过同源变体	以来源和变换族进行外推验证

对照	要排除的替代解释	具体做法
等预算反馈	收益只是多调用模型	固定调用、token 和可见测试预算

这些对照不必一次全部大规模开展。优先顺序是标签可靠性、触发条件、规范一致性、原题级数据划分，再扩展复杂度匹配和已知污染校准。前四项若不成立，后续模型数量越多也无法弥补证据缺陷。

4.5 轨迹采集与对齐的技术注意点

Python 行级追踪事件通常发生在一行执行前；把该时刻的变量状态叫作“执行后状态”会制造系统性错标。需明确采集点语义，并用小程序人工核验分支、循环、返回和异常的时序。

多次执行同一语句必须保留动态序号。引用类型需防止后续原地修改覆盖先前快照；对象表示、集合顺序、浮点 NaN、异常消息、超时等都要有稳定规范。序列化字符串相同不一定语义相同，字符串不同也不一定语义不同。

重命名可通过符号映射对齐；基本块重排、辅助函数抽取或循环变换则可能没有逐行一一对应。此时可在可观察状态检查点上比较，而不是强制匹配全部行序列。无法可靠对齐的变换不宜纳入路径主指标，仍可保留输出级结果。

控制流图用于刻画可能路径，真实执行轨迹是特定输入下的一条动态记录，数据依赖描述值如何流动。报告应分别说明使用静态边、动态边或模型预测边，避免把不同图结构混作同一真值。

4.6 论文写作中建议使用与避免使用的表述

建议：“在已验证的区分输入上，模型出现原答案回退；该现象在控制运行复杂度后仍存在，并在可控暴露实验中与污染条件关联。”避免：“只要变体做错，就证明模型背过原题。”

建议：“过程预测与执行真值存在分歧，说明模型给出的过程答案与该程序运行不一致。”避免：“错误轨迹证明模型内部没有真正推理。”

建议：“在当前测试域未发现行为差异”或“在限定转换规则与语言子集下保持语义”。避免：“通过三个输入，所以两个程序语义等价。”

建议：“相同预算下，隐藏测试上的表现提升，并在未见题目及变换上保留收益。”避免：“看过测试反馈后通过率提高，所以污染问题已经解决。”

五、阅读与复现优先级

第一优先级——直接决定主实验设计：[7] REval、[2] AdvPrompt-MIA、[15] LoCaL、[13] Question Neighbourhood、[5] PatchDiff。需要提取任务定义、数据划分、分母、oracle、采样配置与核心消融，而不只是继续增加文献数量。

第二优先级——确保结论可信：[1] 可控泄漏、[4] 任务对齐、[6] 可复现性、[14] 评价器可靠性；并补齐 [3] 的完整实验。这一组决定哪些样本可以使用、哪些因果表述能够成立。

第三优先级——设计改善与外部验证：[8] TraceCoder、[10] SEER、[11] MGDebugger、[19] AdaptEval；再以 [17][18] 检查效率与现实来源。暂不必同时复制所有训练或多代理框架。

第四优先级——拓展性阅读：[9] VulFaith、[12] 因果辅助解释、[16] ProxyWar、[20] MCR-Bench。前两篇先补全文，后两篇用于动态任务迁移。它们提供有价值的概念，但不是当前三类代码变体实验全部必需的组成部分。

建议形成四份实验资产：带来源与变体关系的数据清单；可重复执行的 oracle 与轨迹采集器；包含原始请求、返回和解析结果的评测日志；按原程序聚类的统计脚本。先在少量人工逐条核验的样本上走通，再扩充规模。

六、文献信息与版本核查索引

以下链接便于追溯：引用正式成果时以期刊 / 会议元数据为准，引用具体实验数值时注明采用的公开稿或终稿。作者过多时使用“等”缩写，不视为完整机器可导入的参考文献格式。

- [1] Hernández López J A, Chen B, Saad M, 等. On Inter-Dataset Code Duplication and Data Leakage in Large Language Models. IEEE TSE, 51(1), 2025: 192–205. [DOI](#)。2024 在线 / 2025 卷期，非两个独立成果。
- [2] Jiang Y, Li Z, Huang S, 等. Effective Code Membership Inference for Code Completion Models via Adversarial Prompts. ASE, 2025. [DOI](#)。公开稿 arXiv:2511.15107。
- [3] Yang Z, Lin H, He Y, 等. Contamination Means Overestimation? A Fine-grained Empirical Study in Code Intelligence. ISSTA, 2026, 已接收. [官方记录](#)。B 级，未获独立全文。
- [4] Wang M, Xu J, He P. PAIChecker: Uncovering and Checking PR-Issue Misalignment in SWE-Bench-Like Benchmarks. ASE, 2026, 已接收. [DOI](#)。公开稿 arXiv:2607.28587。
- [5] Wang Y, Pradel M, Liu Z. Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study. ICSE, 2026. [DOI](#)。本报告采用作者 ICSE 终稿的 6.4 个百分点，不混用早期稿的 6.2。
- [6] Angermeir F, Amougou M, Kreitz M, 等. Reflections on the Reproducibility of Commercial LLM Performance in Empirical Software Engineering Studies. ICSE, 2026. [DOI](#)。初筛总数的版本差异不影响本报告对五项复现结论的限定。
- [7] Chen J, Pan Z, Hu X, 等. Reasoning Runtime Behavior of a Program with LLM: How Far Are We? ICSE, 2025. [官方记录](#)。预印本 arXiv:2403.16437。
- [8] Huang J, Ye W, Sun W, 等. TraceCoder: A Trace-Driven Multi-Agent Framework for Automated Debugging of LLM-Generated Code. ICSE, 2026. [DOI](#)。不要与其他同名 TraceCoder 工作混淆。
- [9] Wang J, Lu J, Yang S, 等. Are Reasoning Traces Faithful in LLM-Based Code Vulnerability Detection? ASE, 2026, 已接收. [官方记录](#)。B 级，未获独立全文。
- [10] Gao S, Wang C, Gao C, Lyu M R. SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning. ICSE, 2026. [公开稿](#)。不要与不相关的同名工具混淆。
- [11] Shi Y, Wang S, Wan C, Wang M, Gu X. From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging. ICSE, 2026. [DOI](#)。预印本编号始于 2024，当前下载版本含更新后的作者与会议页眉。
- [12] Ji Z, Ma P, Li Z, Wang Z, Wang S. Causality-Aided Evaluation and Explanation of Large Language Model-Based Code Generation. PACMSE, 2(ISSTA), 2025: 1374–1397. [DOI](#)。B 级；同主题博士论文章节未作为独立论文全文替代。
- [13] Honarvar S, Rei M, Donaldson A. The “Question Neighbourhood” Approach for Systematic Evaluation of Code-Generating LLMs. IEEE TSE, 51(11), 2025: 3138–3167. [DOI](#)。作者按本次取得的终稿核对。

- [14] Crupi G, Tufano R, Velasco A, 等. On the Effectiveness of LLM-as-a-Judge for Code Generation and Summarization. IEEE TSE, 51(8), 2025: 2329–2345. [DOI](#)。ICSE 2026 Journal First 不重复计数。
- [15] Dristi S B, Dwyer M B. LoCaL: Countering Surface Bias in Code Evaluation Metrics. FSE, 2026. [官方记录](#)。研读公开稿 arXiv:2509.15397 的题名为 Analyzing and Mitigating Surface Bias in Code Evaluation Metrics; 终稿方法差异待逐项比对。
- [16] Peng W, Wang X, Wu Q. ProxyWar: Dynamic Assessment of LLM Code Generation in Game Arenas. ICSE, 2026. [DOI](#)。作者顺序按公开 PDF; 会议人员列表顺序不当作作者排序依据。
- [17] Peng Y, Wan J, Li Y, Ren X. COFFE: A Code Efficiency Benchmark for Code Generation. PACMSE / FSE, 2025. [DOI](#)。公开稿 arXiv:2502.02827。
- [18] Yu X, Chen H, Liu L, 等. RealisticCodeBench: Towards More Realistic Evaluation of Large Language Models for Code Generation. ASE, 2025. [DOI](#)。研读作者提供的正式会议 PDF。
- [19] Zhang T, Mao X, Wang S, 等. AdaptEval: A Benchmark for Evaluating Large Language Models on Code Snippet Adaptation. ASE, 2025. [官方记录](#)。预印本 arXiv:2601.04540, 不据预印本年份改变会议年度。
- [20] Zheng D, Wang Y, Wang X, 等. From Static to Dynamic: Benchmarking Real-World Code Review with MCR-Bench. ISSTA / PACMSE, 2026, 已接收 / 稿内预定 October 卷期. [DOI](#)。公开稿 arXiv:2608.27442; 不是音频等其他领域的同名基准。

七、结语

这组文献支持将本课题定位为“以语义干预和执行证据为基础的诊断研究”。最应避免的是把鲁棒性下降、成员身份、语义错误和自然语言解释失真压缩成一个未经校准的污染分数。

可形成的完整贡献是：可审核的受控变体数据；边界清楚的风险测量；能够定位状态与路径错误的多粒度协议；以及在独立条件下经过验证的改进。即使最终发现某些异常与污染关联不强，只要对照充分，也能形成有价值的研究结论。

本报告完成的是资料核查、重点章节研读、比较分析和实验设计建议。尚未运行上述论文的复现仓库，也未对你的模型结果执行新的统计检验；这些应作为下一阶段工作，而不是写入本报告的既有成果。